Group Number: 18

Group Members: Marta Andres Rodrigues, 20241838@novaims.unl.pt | Leopold Kaiser, 20240296@novaims.unl.pt

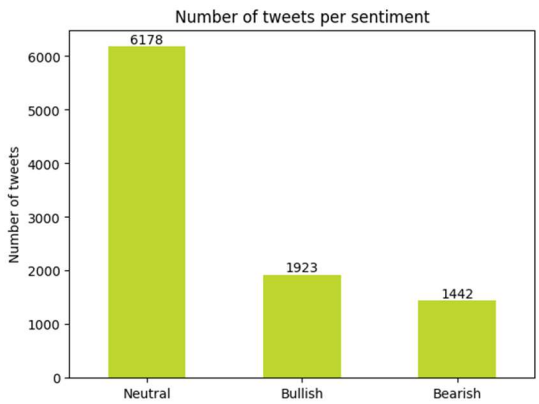
1. Introduction

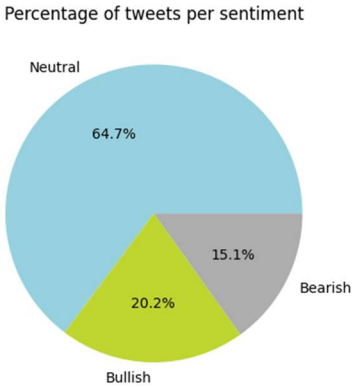
The financial domain is heavily influenced by public sentiment, especially as it relates to stock performance and market trends. This project aims to leverage natural language processing (NLP) and machine learning techniques to classify financial tweets into three sentiment categories: bullish, bearish, and neutral. By combining advanced preprocessing methods with feature engineering models such as TF-IDF, Word2Vec, and Sentence Transformers, we train and evaluate multiple classification algorithms to identify the most effective approach for sentiment analysis in financial contexts. The results provide actionable insights into how sentiment classification can enhance data-driven decision-making in the financial industry.

2. Data Exploration

The corpora is composed of two columns, the tweets and the sentiment, represented by 3 labels: bearish (0), bullish (1) and neutral (2).

The data is imbalanced, most of the tweets can be classified as neutral statements, however, Bullish statements are more represented as bearish statements with a ratio of 20% and 15%, respectively.





Further, one can see that the words most represented are *Https*, *MarketScreener*, *t*, *co* and *Stock*, which tells us that there are a lot of links embedded in the tweets, which will need to be treated in the Data Preprocessing part, and that there are also a lot of mentions to the platform MarketScreener. The word *stock* seem reasonable since most of the comments are related to financial news and information.

3. Data Preprocessing

Several data preprocessing techniques were applied to standardize and clean the data in preparation for the application of the NPL models. This step is crucial in order to remove the noise from the data and help the model better capture the main idea of a sentence.

The procedures tested for preprocessing the data are the following:

- Removing punctuation: cleans the data from punctuation and other special characters;
- Lowercasing: substitutes all capital letters in the text;
- Stop words removal: eliminates words that do not add meaning to the sentence, such as “the”, “and” and others;
- Regular expressions: regex handling is important to treat the patterns in the data. The patterns identified in this corpus are:
    - Prices (e.g. $10, US$2.25,1 and C$50,1), that are substituted with ‘price’;
    - Stocks (e.g. $AMD) that are substituted with ‘stock’;
    - Links (e.g. https://t.co/) that are removed.
- Stemming: reduces the words to their root by removing suffices, which helps reduce the variations of the same word and keeps the core meaning;
- Lemmatization: reduces words to their lemma, which is the base form. This procedure is slower but more accurate than stemming, since it considers grammar rules.

Bonus Points / Extra procedures tested:

- Expanding contractions: expands shortened forms of words (e.g. can’t) to the full form (e.g. cannot) which allows for a more standardized output keeping the text consistent.
- Custom stop word removal: in this procedure, the list of the stop words is altered to remove words that can contain a negative meaning and change the sentiment of the text. The words removed from the stop words list are: "not", "no", "never", "nor”, and "up"

After testing each procedure, the implementation occurred in the following order:

1) Regular expressions, applied first to remove the links and handle the stock and price’s patterns.
2) Expanding contraction, applied second to allow the others procedures to run on standard data.
3) Punctuation.
4) Lowercasing.
5) Custom stop word removal.
6) Lemmatization, instead of stemming, given that the dataset and the sentences aren't too extensive, and it is more accurate.

This is applied to the train and test dataset.

4. Feature Engineering

Feature Engineering is used to transform the text data into data that is understandable to the machine in Natural Language Processing (NLP). All of the following models are applied to the train and test datasets separately.

4.1 BoW

The BoW model transforms the data so that each sentence is represented as a collection of words without considering the structure of the phrase. The model breaks down the sentence into individual words that are treated as features in which the position of each word is discarded.

The model creates a vocabulary from documents that are provided, and each sentence is then represented by a vector composed by the frequency of each word from the vocabulary in that sentence.

This model is simple to understand and implement and is efficient in small scales, however it has its disadvantages, such as, not retaining context, has high dimensionality when the vocabulary is large, similar words aren't aggregated.

4.2 TF-IDF

Term Frequency-Inverse Document Frequency is another NPL model which adjusts the frequency of words in a sentence based on the importance of given word in the whole dataset, providing a score for each word. Higher scores mean that the word is important in identifying the content of the sentence / document.

The TF is the proportion between the number of times the word appears in a sentence over the total number of words in the dataset, measuring how often a word appears in a sentence. IDF is the log of the total number of sentences over the number of sentences that contain word, measures the importance of each word in the entire corpus, showing that a word that appears in a lot of documents / sentences, is not very meaningful. The final score is given by one measure multiplied by the other.

This model is an improvement from BoW since it also accounts for the importance of each word and not just the occurrence.

4.3 Word to vector

The Word to Vector (Word2Vec) feature engineering method transforms textual data into numerical representations that capture semantic relationships between words. By leveraging the Skip-gram or Continuous Bag of Words (CBOW) models, Word2Vec learns to predict word contexts and assigns vector embeddings to words in a high-dimensional space. This enables the model to encode meaningful relationships, such as synonyms or contextual similarity, trough the closeness in the vector space since similar words have similar word vectors, unlike BoW and TF-IDF.

In the context of this project, Word2Vec was used to represent the sentences by aggregating the vector representations of individual words. This approach allows for the capture of nuanced

linguistic patterns, aiding downstream machine learning models in tasks like sentiment classification. However, its effectiveness depends on the quality of the input data and preprocessing, as well as the balance and size of the dataset.

> Bonus Point/ Extra NPL models tested:

4.4 Sentence to vector

The sentence-to-vector model is a powerful approach for transforming sentences into dense numerical representations preserving the semantic meaning of the sentences by mapping them to a fixed-dimensional vector space. Such representations are particularly useful for a wide range of natural language processing tasks, including classification.

The model employs the Doc2Vec algorithm, an extension of Word2Vec, incorporating a unique identifier for each sentence during training, allowing the model to learn a vector representation that encapsulates the entire sentence's context.

Sentence vectors can be used to measure semantic similarity between sentences using metrics like cosine similarity. High similarity scores indicate that the sentences share a common meaning or theme, even if their wording differs.

Beyond similarity analysis, sentence vectors are invaluable for tasks such as text classification such as this project, where they serve as input features for the machine learning models.

The sentence-to-vector model offers significant advantages over traditional techniques like Bag of Words or TF-IDF by considering word order and context, resulting in richer and more nuanced representations. However, its performance is influenced by factors such as the chosen hyperparameters, including `vector_size`, `window`, and `min_count`.

4.5 Sentence Transformer

The Sentence Transformer generates dense vector representations for sentences, capturing their semantic meaning. In this project, the pre-trained `'all-MiniLM-L6-v2'` model is employed to convert a list of cleaned sentences into their corresponding embeddings.

One of the key benefits of using this model is its ability to capture semantic relationships between sentences, allowing for a more nuanced understanding of the underlying sentiments in news articles. Additionally, the model generates dense, lower-dimensional vectors instead that are better for performance.

Leveraging pre-trained models like `'all-MiniLM-L6-v2'`, one can save considerable time and computational resources.

5. Classification Models

After using NLP models to convert the cleaned text data, the Classification Models are Machine Learning algorithms that predict the category, in this case, the sentiment, for the given input data.

These models are trained on each training dataset produced by the NLP models. Afterwards they are applied to the test dataset, that is made up by 25% of the original data, and evaluated based on accuracy, precision, recall and F1-score.

5.1 KNN

The KNN model classifies a data point, in this case, a sentence, based on label that is most common in the 'k' nearest neighbours in the feature space, using, in this case, the cosine distance.

In the context of feature engineering, calculating cosine similarity between sentence embeddings is crucial. By using the `cosine_similarity` function from the `sklearn.metrics.pairwise` module, the cosine similarity between all pairs of sentence embeddings is computed. The resulting similarity matrix provides a quantitative measure of the semantic similarity between sentences. This measure of semantic similarity aids in identifying sentences that are semantically similar, which is vital for accurately classifying new comments into categories such as neutral, bearish, and bullish.

The KNN model is non-parametric which makes no assumptions about the data distribution and works well in smaller datasets.

The results of this model highly depend on the hyperparameter k, and for that reason three different versions are trained, k=5, k=10 and k=20.

5.2 Logistic Regression

Logistic regression is a model used for binary classification, however in our case when have 3 classes (Bearish, Bullish and Neutral). This model aims to predict the probability for a sentence to belong to a class and then, using the sigmoid function, classifies the sentence.

For multiple class dataset, it uses the softmax function to calculate the probabilities of each class, choosing the class with the highest probability.

5.3 Multi-Layer Perceptron

The Multi-Layer Perceptron is feedforward neural network used in classification and regression tasks, where the data flows through hidden layers composed by neurons, that process data by applying activation functions that enables the model to learn complex patterns.

During the training of this model, the connections between neurons have a weight that determines its importance, and each neuron has a bias that are adjusted and optimized, in our case using an optimization algorithm, such as gradient descent. The activation function sets the output of each neuron that are passed to the next layer. When the data reaches the output layer, it normally has a neuron per class, and the output is passed using the softmax function with the probability distribution per class. Using backpropagation, the weights and biases are adjusted to minimize the error through gradient descent.

The model applied in this project uses the *adam* solver since the size of the training dataset is considerable. For the hidden layers, 2 hidden layers were defined, having 2 neurons each, for high dimensionality input vectors and, and 3 hidden layers, with 3 neuros each for lower dimensionality input vectors, such as the ones provided by word2vec, sentence2vec and Sentence Transformer.

While the MLP model is powerful, modelling non-linear relationships between input and output, and is also able to learn the features and patterns in the data, this model is computationally expensive and also sensitive to hyperparameters (activation function, number of hidden layers and number of neurons). Only one combination of hidden layers was tested for each input given the computational load.

5.4 Naïve Bayes

The Naïve Bayes classifier is based on the Bayes' Theorem and used in text classification tasks given how fast it is to implement even on large datasets and how well it works despite the naïve categorization.

This classifier is based on Bayes' Theorem that finds the probability of an event occurring given the probability of another event that has already occurred and assumes that the features used in a sentence are independent of each other, hence the naïve nomination.

Despite these assumptions, this model has worked very well in real word scenarios even with small training data. One disadvantage is that it assigns zero probability to unseen events, leading to poor results when working with new data.

Since we have 3 classes, the MultinomialNB model is used, assuming that words / features follow a multinomial distribution. The classifier calculates the probability of each sentence belonging to each class based on the frequency of the words.

> Bonus Points / Extra classifiers tested:

5.5 Long Short-Term Memory

Long Short-Term Memory (LSTM) model is a type of recurrent neural network (RNN) designed to effectively capture long-term dependencies in sequential data, making them particularly suitable for natural language processing (NLP) tasks like sentiment analysis. In this project, LSTM is used to analyze financial news articles by leveraging their ability to process sequential word embeddings and retain context across longer textual spans. Unlike traditional RNNs, LSTM mitigates the vanishing gradient problem through specialized memory cells and gating mechanisms, which allow the model to selectively forget, remember, and update information. By processing text data transformed into numerical representations such as word embeddings, the LSTM model identifies patterns that contribute to sentiment classification. Its ability to handle context-dependent nuances enhances the model's performance, especially when analyzing complex financial language, contributing to robust sentiment prediction for bullish, bearish, or neutral categories.

5.6 Fine-tuned BERT

Bidirectional Encoder Representations from Transformers (BERT) is a pretrained language model that was devoted by Google aiming to understand the context of words in a sentence looking before and after the word in the sentence, using the Transformer architecture. By using this model, we are leveraging the training already done on large corpus, such as *Wikipedia*.

To make the model adjusted to our data, we took the "yiyanghkust/finbert-tone" model, which is already pre-trained on financial communication text, label as positive, negative or neutral, from analyst reports, and fine-tuned it further using our dataset. We map the positive label as bullish and the negative label as bearish.

Some alterations had to be done to follow the model's assumptions, such as change the Neutral label to 0 and the Bearish label to 2. Also, no data cleaning method was applied in the fine tuning of the model since the model had extensive training to capture the context of the sentence. A validation dataset was also created to use in the training of the fine-tuned version to avoid overfitting to the training data.

Since 2018 BERT has been a state-of-the-art model in natural language processing tasks, by fine tuning this model we expect to obtain the best results in the classification of the tweets as Bearish, Bullish and Neutral.

6. Evaluation and Results

The 31 models (5 Encoders * 7 Classifiers + 1 LSTM + 1 FineTuned Bert) were evaluated based on 4 measures: accuracy, precision, recall and F1-score.

Given that the purpose is to describe the sentiment behind the tweets holding financial information and that it may influence investment decisions, it is important to achieve a high precision score, which shows the percentage of how many of the predictions for each class were actually correct, since misclassifying a Bearish tweet can lead to investors selling their positions while the asset is still increasing in valuation, and misclassifying a Bullish tweet can lead to investors buying an asset that is depreciating or losing investment opportunities in other assets.

The accuracy is also important to take into account since it captures how much of the data the model predicted correctly. As for the recall score, it measures, for each class, how many predictions were correct. The F1-score is a measure that weights the precision and recall measure into one.

To visualize each score for each model, a summary table for each measure was created:

Classifier	KNN10	KNN20	KNN5	LR	MLP	NB	RF
Encoding							
bow	0.752305	0.744342	0.751048	0.795474	0.682733	0.618189	0.777871
sentence2vector	0.685666	0.687343	0.670997	0.667225	0.658843	0.576278	0.660520
sentence_transformer	0.761106	0.752305	0.754820	0.772842	0.760687	0.672674	0.694468
tfidf	0.770746	0.766136	0.755658	0.778290	0.745180	0.684828	0.768650
w2v	0.639564	0.653395	0.611484	0.637888	0.637888	0.486169	0.642079

Table 1.1 - Accuracy Scores

As shown in table 1.1, the model that achieves the highest score that capture the overall number of correct predictions was BoW and Logistic Regression (0.795) which is slightly unexpected since BoW is a simple method of encoding text, not keeping the context of the sentence. The second-best model was TF-IDF and Logistic Regression with an accuracy of 77.8% correctly classified tweets.

From the classifiers, Logistic Regression showed the best results given all the encoders used, with 3 of the results being in the top 5 of the best accuracies obtained. As for the encoders, BoW and TF-IDF, the simplest methods used, performed the best.

The method Sentence 2 Vector and word2vec surprisingly returned very low accuracy scores. This can result from the aggregation of vectors to form a single sentence vector, which might lose the context needed for sentiment analysis in shorter sentences like tweets.

Sentence Transformers takes word importance and context which results in better overall results in all the classifiers tested, resulting in the 4th best model when used with Logistic Regression.

Precision Scores for Bearish Class (Precision Measure)

Classifier / Encoding	KNN10	KNN20	KNN5	LR	MLP	NB	RF
bow	0.697297	0.794872	0.610487	0.715953	0.311615	0.336222	0.792899
sentence2vector	0.521008	0.555556	0.415730	0.448819	0.392045	0.278752	0.593750
sentence_transformer	0.680000	0.726776	0.638889	0.686992	0.580460	0.492823	0.861111
tfidf	0.717172	0.791367	0.623431	0.840000	0.367816	1.000000	0.721311
w2v	0.361446	0.461538	0.248619	0.000000	0.000000	0.159091	0.615385

Precision Scores for Bullish Class (Precision Measure)

Classifier / Encoding	KNN10	KNN20	KNN5	LR	MLP	NB	RF
bow	0.645833	0.680115	0.638889	0.726415	0.566308	0.467091	0.724490
sentence2vector	0.540070	0.602679	0.494536	0.459016	0.416431	0.369932	0.502857
sentence_transformer	0.616052	0.621827	0.588803	0.707895	0.681818	0.457060	0.823529
tfidf	0.682500	0.713514	0.626068	0.740437	0.701835	0.821705	0.707500
w2v	0.418972	0.511905	0.388102	0.000000	0.000000	0.251289	0.465909

Precision Scores for Neutral Class (Precision Measure)

Classifier / Encoding	KNN10	KNN20	KNN5	LR	MLP	NB	RF
bow	0.780407	0.752862	0.802015	0.824633	0.815593	0.792975	0.787945
sentence2vector	0.716667	0.700957	0.730727	0.713920	0.730210	0.790788	0.674162
sentence_transformer	0.811176	0.783306	0.826114	0.798864	0.819753	0.846995	0.683830
tfidf	0.796421	0.774640	0.810602	0.780749	0.838951	0.675712	0.787022
w2v	0.678049	0.666515	0.689525	0.637888	0.637888	0.653417	0.649015

Table 1.2- Precision Scores

The precision score measures the reliability of the predictions in each class, which is the number of times the model correctly predicted the bearish tweets from all the bearish tweets in the testing data, and so on for the remaining classes (bullish and neutral).

As stated previously, this measure is very important in this context since it might influence investment decisions and wrong classifications can lead to investors losing money or losing out on money (when tweets are incorrectly marked as bearish), or asset managers making poor portfolio management decisions, for example, by mistakenly expecting market optimism if tweets are wrongly classified as bullish.

For this score, we must look closely at the bearish and bullish classes. The TF-IDF + Naïve Bayes model returns incredible results correctly classifying all the bearish tweets and being the second-best in classifying the bullish tweets (82.2% correctly classified) however it did not perform as well for the neutral class (0.676) which means that the model tends to classify tweets as neutral when they are not. Despite the low accuracy score of 0.685, this is a conservative approach for our context since it does not suggest investment decisions for some tweets, which makes it a good model for low-risk investors.

The model composed by Sentence Transformer and Random Forest shows the second-best score for precision in the bearish class (0.861) and shows the best performance for bullish tweets (0.824), however it still did not perform as well in the neutral class (0.684).

The models with the highest scores in the neutral class, show lower scores in the bullish and bearish class which is a problem in this financial context.

The overall best model, given the 3 classes, is the TF-IDF + LR model, with the third-best score in the bearish class (0.840) and in the bullish class (0.740) and a score of 0.781 in the neutral class. For this reason, this model is a strong candidate for the best overall model, since it also has the second-best accuracy.

Strangely, the models taking word2vec vectors as an input perform very poorly, especially when combined with Linear Regression and Multi-Layer Perceptron, where the models are not able to classify any bearish or bullish tweets correctly, which defeats the purpose of these classification models.

Recall Scores for Bearish Class (Recall Measure)

Classifier / Encoding	KNN10	KNN20	KNN5	LR	MLP	NB	RF
bow	0.344920	0.248663	0.435829	0.491979	0.294118	0.518717	0.358289
sentence2vector	0.165775	0.106952	0.197861	0.152406	0.184492	0.382353	0.050802
sentence_transformer	0.409091	0.355615	0.430481	0.451872	0.540107	0.550802	0.082888
tfidf	0.379679	0.294118	0.398396	0.336898	0.342246	0.024064	0.352941
w2v	0.080214	0.032086	0.120321	0.000000	0.000000	0.074866	0.021390

Recall Scores for Bullish Class (Recall Measure)

Classifier / Encoding	KNN10	KNN20	KNN5	LR	MLP	NB	RF
bow	0.506122	0.481633	0.563265	0.628571	0.644898	0.448980	0.579592
sentence2vector	0.316327	0.275510	0.369388	0.285714	0.300000	0.446939	0.179592
sentence_transformer	0.579592	0.500000	0.622449	0.548980	0.581633	0.640816	0.228571
tfidf	0.557143	0.538776	0.597959	0.553061	0.624490	0.216327	0.577551
w2v	0.216327	0.175510	0.279592	0.000000	0.000000	0.397959	0.083673

Recall Scores for Neutral Class (Recall Measure)

Classifier / Encoding	KNN10	KNN20	KNN5	LR	MLP	NB	RF
bow	0.931669	0.950723	0.888962	0.923784	0.790407	0.697109	0.944809
sentence2vector	0.932326	0.962549	0.884363	0.916557	0.890933	0.665572	0.965177
sentence_transformer	0.906045	0.931012	0.877135	0.923784	0.872536	0.712878	0.994744
tfidf	0.935611	0.955322	0.894218	0.959264	0.883049	0.998029	0.932326
w2v	0.913272	0.959921	0.839028	1.000000	1.000000	0.615637	0.974376

Table 1.3 - Recall Scores

The recall score is the second priority in the analysis of the model in a financial context of our objective, however it is relevant in risk management to capture early warning. This score shows how many of the tweets that were classified as bearish are actually bearish, and so on for the other classes.

When a model fails to identify bearish tweets, it can make the investor or financial analyst miss early warnings such as negative news impacting a sector. For this reason it is important to have an high score, especially in the bearish class.

In the TF-IDF + LR model, the recall score for bearish is very low (0.337) which means the model doesn't perform as well in capturing all the bearish tweets, however, base on the precision scorre, the ones it captures are correct. This is also correct for the bullish tweets, with a score of (0.553). This makes the model more conservative which is a positive attribute in this context.

The recall score are overall low for the bearish and bullish class, but that does not apply for the neutral tweets which might be a result of the imbalanced data used in the training process. To improve classification for less frequent classes, strategies such as balancing the dataset or using advanced classification methods could be explored.

The F1-score is a mix of the precision and recall measure, which reenforces the ideia that the TF-IDF + LR model is the best model for our scope with 0.481 for the bearish class (the maximum score in this class is 0.583), 0.633 in the bullish class (5th best model) and 0.861 in the neutral class (2nd best model).

F1-Score for Bearish Class (F1-Score Measure)

Classifier / Encoding	KNN10	KNN20	KNN5	LR	MLP	NB	RF
bow	0.461538	0.378819	0.508580	0.583201	0.302613	0.407992	0.493554
sentence2vector	0.251521	0.179372	0.268116	0.227545	0.250909	0.322435	0.093596
sentence_transformer	0.510851	0.477558	0.514377	0.545161	0.559557	0.520202	0.151220
tfidf	0.496503	0.428850	0.486134	0.480916	0.354571	0.046997	0.473968
w2v	0.131291	0.060000	0.162162	0.000000	0.000000	0.101818	0.041344

F1-Score for Bullish Class (F1-Score Measure)

Classifier / Encoding	KNN10	KNN20	KNN5	LR	MLP	NB	RF
bow	0.567506	0.563919	0.598698	0.673961	0.603053	0.457856	0.643991
sentence2vector	0.398970	0.378151	0.422897	0.352201	0.348754	0.404806	0.264662
sentence_transformer	0.597266	0.554299	0.605159	0.618391	0.627753	0.533560	0.357827
tfidf	0.613483	0.613953	0.611691	0.633178	0.660907	0.342488	0.635955
w2v	0.285330	0.261398	0.325030	0.000000	0.000000	0.308057	0.141869

F1-Score for Neutral Class (F1-Score Measure)

Classifier / Encoding	KNN10	KNN20	KNN5	LR	MLP	NB	RF
bow	0.849356	0.840302	0.843253	0.871398	0.802803	0.741958	0.859277
sentence2vector	0.810394	0.811185	0.800238	0.802647	0.802604	0.722797	0.793840
sentence_transformer	0.855990	0.850796	0.850860	0.856795	0.845321	0.774171	0.810493
tfidf	0.860423	0.855546	0.850359	0.860849	0.860435	0.805836	0.853534
w2v	0.778275	0.786753	0.756965	0.778915	0.778915	0.633965	0.779091

Table 1.4 - F1-Score

Given all the metrics analysed so far, with the reasons previously stated, the best model to be chosen would be TF-IDF as the encoder and Logistic Regression as the classifier, having the best overall

scores so far. However, as expected, by fine tuning the pretrained FinBERT model, we were able to achieve the best results so far, with an accuracy of 87%, an increase of 917.1 b.p. from the TF-IDF + LR model, and overall better precision and recall scores for all the classes, consequently better F1-scores as well.

The results for the LSMT model, as expected for higher complexity models, returns also high accuracy of 76%, however it still falls behind the LLM finBERT. When looking at the precision scores, they are significantly lower than finBERT, with the bearish and bullish scores below 0.65 and recall score of 0.56 for bearish tweets which can have negative impacts on the financial decisions made using this model.

	precision	recall	f1-score	support
Negative	0.54	0.56	0.55	374
Positive	0.63	0.70	0.66	490
Neutral	0.86	0.83	0.84	1522
accuracy			0.76	2386
macro avg	0.68	0.69	0.69	2386
weighted avg	0.76	0.76	0.76	2386

Figure 1.5 - Results using LSTM model

For the finetuned finBERT model, precision score for the bullish class is 0.81, which is 135.3 b.p. better than the best model for this class and measure (Sentence Transformer + Random Forest). The precision score for the bearish class is 0.73, which is considerably worse than the best model for this class and measure, however the recall score is much better than the best model previously tested, meaning that it is good for detecting early warnings. For the neutral class, the precision score of 0.92 is 730 b.p. better than the best model tested for this class.

Results using the Fined-tuned FinBERT

	precision	recall	f1-score	support
Neutral	0.92	0.91	0.91	1537
Bullish	0.81	0.82	0.82	484
Bearish	0.73	0.75	0.74	365
accuracy			0.87	2386
macro avg	0.82	0.83	0.82	2386
weighted avg	0.87	0.87	0.87	2386

Results using "yiyanghkust / FinBERT-tone"

	precision	recall	f1-score	support
Neutral	0.86	0.78	0.82	1680
Bullish	0.58	0.68	0.62	420
Bearish	0.47	0.62	0.54	286
accuracy			0.74	2386
macro avg	0.64	0.69	0.66	2386
weighted avg	0.76	0.74	0.75	2386

Table 1.6 – Results using FinBert model

By comparing our fine-tuned version of the model with the version we trained on, its clear the importance and impact of further training the data on the context we want to apply it to. By doing so we were able to improve the performance of FinBERT-tone by 1300 b.p. in accuracy, and also the precision, recall and f1-score measures also improved immensely on classes with lower representation such as the bearish and bullish class, which leaves us to conclude that the issue with imbalanced data is able to be overcome with adapting the BERT model to a given context.

Having chosen the fine-tuned FinBERT-tone model as the best model, it will be applied to the test.csv dataset for the final submission.